



Phase 2

Part A — Functions

A **function** is a reusable block of code that performs a specific task. Instead of writing the same logic over and over, you write it once inside a function and call it whenever needed.

Think of a function as a **machine**: you give it input → it does something → it gives you output.

1. Why Do We Need Functions?

Without functions:

```
let length1 = 5, breadth1 = 3;
let area1 = length1 * breadth1;
console.log(area1);

let length2 = 10, breadth2 = 4;
let area2 = length2 * breadth2;
console.log(area2);

let length3 = 7, breadth3 = 2;
let area3 = length3 * breadth3;
console.log(area3);
```

With a function:

```
function calculateArea(length, breadth) {
  return length * breadth;
}
```

```
console.log(calculateArea(5, 3)); // 15
console.log(calculateArea(10, 4)); // 40
console.log(calculateArea(7, 2)); // 14
```

Three big benefits:

1. **Reusability** — write once, use many times.
 2. **Readability** — `calculateArea(5, 3)` tells you what's happening; `5 * 3` doesn't.
 3. **Maintainability** — fix a bug in one place, not ten.
 4. Accessibility
-

2. Function Declaration

The classic syntax:

```
function greet(name) {
  console.log("Hello, " + name);
}

greet("Aman"); // Hello, Aman
greet("Priya"); // Hello, Priya
```

Anatomy:

- `function` — the keyword
- `greet` — the function name
- `(name)` — parameter list (inputs)
- `{ ... }` — function body (the code)

Parameters vs Arguments:

- **Parameter** = the placeholder in the definition (`name`)
- **Argument** = the actual value passed when calling (`"Aman"`)

Most people use the words interchangeably — that's fine — but technically they're different.

3. Function Expression

A function can also be stored in a variable.

```
const greet = function(name) {  
  console.log("Hello, " + name);  
};  
  
greet("Aman");
```

Notice the semicolon at the end — because it's an assignment statement.

Key difference from declarations:

- Function declarations are **hoisted** (you can call them before they're written — we'll explore this in Phase 3).
- Function expressions are **not** hoisted in the same way.

For now, just know both exist and both work.

4. Anonymous Functions

A function without a name is called **anonymous**. The function expression above is anonymous — it has no name; it's just stored in a variable.

```
const sayHi = function() {  
  console.log("Hi!");  
};
```

Anonymous functions are very commonly used as **callbacks** (we'll see this soon).

5. Arrow Functions (Modern, Preferred)

ES6 introduced a shorter syntax called **arrow functions**.

```
// Regular function expression
const add = function(a, b) {
  return a + b;
};

// Arrow function – same thing
const add = (a, b) => {
  return a + b;
};
```

Shorthand rules:

```
// Single expression → implicit return (no braces, no return keyword)
const add = (a, b) => a + b;

// Single parameter → parentheses optional
const square = x => x * x;

// No parameters → empty parentheses required
const greet = () => console.log("Hello");

// Multi-line body → braces and explicit return required
const add = (a, b) => {
  const sum = a + b;
  return sum;
};
```

When to use arrow vs regular functions?

- For short, single-purpose helpers → arrow functions.
- For methods inside objects, or anywhere you need `this` → regular functions (we'll cover `this` in Phase 4).
- For top-level "main" functions in a file → either works; team style decides.

6. Parameters: Defaults, Rest, and Edge Cases

Default Parameters

If no argument is passed, use a default value:

```
function greet(name = "Guest") {  
  console.log("Hello, " + name);  
}  
  
greet("Aman");    // Hello, Aman  
greet();          // Hello, Guest
```

Rest Parameters

Sometimes you don't know how many arguments will be passed. Use `...` to collect them all into an array:

```
function sum(...numbers) {  
  let total = 0;  
  for (let n of numbers) {  
    total += n;  
  }  
  return total;  
}  
  
console.log(sum(1, 2, 3));    // 6  
console.log(sum(1, 2, 3, 4, 5, 6)); // 21
```

`numbers` is a real array here — you can use any array method on it.

What If You Pass Too Many or Too Few?

JavaScript doesn't complain. Missing parameters become `undefined`; extra arguments are simply ignored.

```
function add(a, b) {
  return a + b;
}

console.log(add(5));           // NaN (5 + undefined)
console.log(add(5, 10, 20)); // 15 (the 20 is ignored)
```

This is *very* different from languages like Java or Python. Use defaults or rest parameters to handle it gracefully.

7. Return Values

A function can **return** a value back to whoever called it.

```
function multiply(a, b) {
  return a * b;
}

let result = multiply(4, 5);
console.log(result); // 20
```

Rules about `return`:

- A function can have **multiple return statements** — but only one will actually execute (the first one reached).
- `return` immediately exits the function — code after it doesn't run.
- A function without `return` returns `undefined` by default.

```
function checkAge(age) {
  if (age < 0) return "Invalid age"; // early return
  if (age >= 18) return "Adult";
  return "Minor";
}

console.log(checkAge(-5)); // "Invalid age"
```

```
console.log(checkAge(20)); // "Adult"
console.log(checkAge(10)); // "Minor"
```

8. Functions as First-Class Citizens

In JavaScript, functions are **values**, just like numbers or strings. This means you can:

1. Store functions in variables.
2. Pass functions as arguments to other functions.
3. Return functions from other functions.

```
// 1. Store in a variable
const sayHi = function() { console.log("Hi"); };

// 2. Pass as argument
function callTwice(fn) {
    fn();
    fn();
}
callTwice(sayHi); // Hi Hi

// 3. Return from a function
function makeGreeter(greeting) {
    return function(name) {
        console.log(greeting + ", " + name);
    };
}
const helloGreeter = makeGreeter("Hello");
helloGreeter("Aman"); // Hello, Aman
```

This is one of JavaScript's most powerful features. We'll use it constantly.

9. Callback Functions

A **callback** is a function that is passed as an argument to another function, to be called later.

```
function processUser(name, callback) {  
  console.log("Processing user: " + name);  
  callback(name);  
}  
  
function welcome(name) {  
  console.log("Welcome, " + name);  
}  
  
processUser("Aman", welcome);  
// Processing user: Aman  
// Welcome, Aman
```

Real-world example you've already seen: `setTimeout`.

```
setTimeout(function() {  
  console.log("3 seconds passed");  
}, 3000);
```

Here, the anonymous function is a **callback** that `setTimeout` calls after 3000ms.

Why callbacks matter: They are everywhere in JavaScript — array methods, event handlers, async operations. Get comfortable with them now.

10. Higher-Order Functions

A **higher-order function** is a function that either:

- Takes another function as an argument, OR
- Returns a function.

Both `processUser` and `makeGreeter` above are higher-order functions.

This isn't a new concept — it's just the *name* for the pattern you just learned.

11. IIFE — Immediately Invoked Function Expression

An IIFE is a function that runs **the moment it's defined**.

```
(function() {  
    console.log("I run immediately!");  
})();
```

Why the extra parentheses?

- Wrapping the function in `()` turns it into an expression.
- The final `()` calls it.

Why use IIFE? Mainly to create a private scope so variables don't pollute the global scope. Modern code uses modules instead, but you'll still see IIFEs in older codebases and interviews.

12. Pure vs Impure Functions

Pure function:

- Given the same input, always returns the same output.
- No side effects (doesn't change anything outside itself).

```
function add(a, b) {  
    return a + b;  
}  
// Pure: add(2, 3) is always 5. Nothing outside changes.
```

Impure function:

- Output may vary, OR it changes something outside (modifies a global variable, logs to

```
let total = 0;  
function addToTotal(n) {  
    total += n;           // side effect – changes outer variable
```

```
    return total;
}
```

Why care? Pure functions are easier to test, debug, and reason about. Prefer pure functions whenever possible.

13. Recursion (Basics)

A function that calls **itself** is called recursive.

```
function factorial(n) {
    if (n <= 1) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}

console.log(factorial(5)); // 120
```

Every recursive function needs:

1. A **base case** — when to stop.
2. A **recursive case** — calling itself with a smaller problem.

Without a base case, you get **infinite recursion** → stack overflow.

We'll see more recursion in Phase 3 when we discuss the call stack.

Part B — Arrays

An **array** is an ordered list of values. It's how we store collections in JavaScript.

1. Creating Arrays

```
let fruits = ["apple", "banana", "mango"];
let numbers = [1, 2, 3, 4, 5];
let mixed = ["Aman", 25, true, null]; // arrays can hold any types
let empty = [];
```

You can also use the `Array` constructor, but the literal syntax `[]` is preferred.

2. Accessing Elements (Zero-Indexed)

Arrays start at index **0**.

```
let fruits = ["apple", "banana", "mango"];

console.log(fruits[0]); // "apple"
console.log(fruits[1]); // "banana"
console.log(fruits[2]); // "mango"
console.log(fruits[3]); // undefined (doesn't exist)
```

Negative indices don't work like in Python. `fruits[-1]` is `undefined`.

To get the last element:

```
console.log(fruits[fruits.length - 1]); // "mango"
console.log(fruits.at(-1));             // "mango" (modern,
cleaner)
```

3. Array Length

```
let arr = [10, 20, 30, 40];
console.log(arr.length); // 4
```

`length` is always **one more than the last index**.

You can even set `length` to truncate an array (rarely useful but worth knowing):

```
arr.length = 2;
console.log(arr); // [10, 20]
```

4. Mutating Methods (Change the Original)

These methods **modify** the array in place.

Method	What it does
<code>push(x)</code>	Adds to the end
<code>pop()</code>	Removes from the end (and returns it)
<code>unshift(x)</code>	Adds to the beginning
<code>shift()</code>	Removes from the beginning (and returns it)
<code>splice(start, count, ...items)</code>	Add/remove anywhere
<code>reverse()</code>	Reverses in place
<code>sort()</code>	Sorts in place

```
let arr = [2, 3, 4];

arr.push(5);           // [2, 3, 4, 5]
arr.unshift(1);        // [1, 2, 3, 4, 5]
let last = arr.pop();  // [1, 2, 3, 4], last = 5
let first = arr.shift(); // [2, 3, 4], first = 1

arr.reverse();         // [4, 3, 2]
arr.sort();            // [2, 3, 4]
```

`splice` — the swiss army knife:

```
let arr = [1, 2, 3, 4, 5];

// Remove 2 elements starting from index 1
arr.splice(1, 2);    // arr is now [1, 4, 5]

// Insert without removing: count = 0
let arr2 = [1, 2, 5];
arr2.splice(2, 0, 3, 4); // arr2 is now [1, 2, 3, 4, 5]

// Replace: remove + insert
let arr3 = [1, 2, 99, 4];
arr3.splice(2, 1, 3);  // arr3 is now [1, 2, 3, 4]
```

Important gotcha with `sort()`:

By default, `sort` converts everything to strings and sorts alphabetically.

```
[10, 1, 5, 100].sort();    // [1, 10, 100, 5] ← surprising!
```

For numeric sorting, pass a compare function:

```
[10, 1, 5, 100].sort((a, b) => a - b);    // [1, 5, 10, 100]  
ascending  
[10, 1, 5, 100].sort((a, b) => b - a);    // [100, 10, 5, 1]  
descending
```

5. Non-Mutating Methods (Return New Array)

These **don't** change the original — they return a new array or value.

```
let arr = [1, 2, 3, 4, 5];  
  
arr.slice(1, 4);           // [2, 3, 4] (original unchanged)  
arr.concat([6, 7]);        // [1, 2, 3, 4, 5, 6, 7]  
arr.includes(3);           // true  
arr.indexOf(3);            // 2  
arr.indexOf(99);           // -1 (not found)  
arr.join("-");             // "1-2-3-4-5"
```

Critical distinction: `slice` (non-mutating, makes a copy) vs `splice` (mutating, modifies original). They look similar but are very different. This trips up everyone.

6. Iteration Methods (The Powerful Ones)

These methods take a **callback** and run it for each element. This is where Phase 2 gets exciting.

forEach — just run a function on each element

```
let nums = [1, 2, 3];
nums.forEach(function(n) {
  console.log(n * 2);
});
// 2, 4, 6
```

With arrow function:

```
nums.forEach(n => console.log(n * 2));
```

The callback can also receive **index** and the **whole array**:

```
nums.forEach((value, index, array) => {
  console.log(index, value);
});
```

`forEach` does NOT return anything (it returns `undefined`). It's just for side effects.

map — transform each element into something new

```
let nums = [1, 2, 3, 4];
let doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8]
console.log(nums); // [1, 2, 3, 4] ← unchanged
```

`map` returns a **new array of the same length**.

filter — keep only elements that pass a condition

```
let nums = [1, 2, 3, 4, 5, 6];
let evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6]
```

`filter` returns a new array, possibly shorter.

reduce — boil the array down to a single value

```
let nums = [1, 2, 3, 4];
let sum = nums.reduce((acc, n) => acc + n, 0);
console.log(sum); // 10
```

- `acc` = accumulator (the running result)
- `n` = current element
- `0` = initial value of acc

Step-by-step trace:

- Start: acc = 0
- Step 1: acc = 0 + 1 = 1
- Step 2: acc = 1 + 2 = 3
- Step 3: acc = 3 + 3 = 6
- Step 4: acc = 6 + 4 = 10

`reduce` is the most powerful — and the most confusing at first. Give students lots of practice.

find — return the first matching element

```
let users = [{name: "A", age: 20}, {name: "B", age: 30}];
let user = users.find(u => u.age > 25);
console.log(user); // {name: "B", age: 30}
```

findIndex — return the index of the first match

```
let nums = [10, 20, 30, 40];
let idx = nums.findIndex(n => n > 25);
console.log(idx); // 2
```

some — does AT LEAST ONE match?

```
let nums = [1, 2, 3];
console.log(nums.some(n => n > 2)); // true
console.log(nums.some(n => n > 10)); // false
```

every — do ALL match?

```
let nums = [1, 2, 3];
console.log(nums.every(n => n > 0)); // true
console.log(nums.every(n => n > 2)); // false
```

7. Array Destructuring

A clean way to extract values from arrays into variables.

```
let arr = [10, 20, 30];

// Old way
let a = arr[0];
let b = arr[1];

// New way
let [x, y, z] = arr;
console.log(x, y, z); // 10 20 30
```

Skip elements:

```
let [first, , third] = [1, 2, 3];
console.log(first, third); // 1 3
```

Default values:

```
let [a = 10, b = 20] = [5];
console.log(a, b); // 5 20
```


Swap variables (elegant!):

```
let x = 1, y = 2;  
[x, y] = [y, x];  
console.log(x, y);    // 2 1
```

8. Spread and Rest with Arrays

Both use `...`, but they do **opposite** things.

Spread — expand an array

```
let nums = [1, 2, 3];  
let more = [0, ...nums, 4];  
console.log(more);    // [0, 1, 2, 3, 4]  
  
// Copy an array (shallow)  
let copy = [...nums];  
  
// Combine arrays  
let combined = [...[1, 2], ...[3, 4]];    // [1, 2, 3, 4]  
  
// Pass as function arguments  
console.log(Math.max(...[5, 3, 9, 1]));    // 9
```

Rest — collect into an array

```
function sum(...nums) {  
    return nums.reduce((a, b) => a + b, 0);  
}  
console.log(sum(1, 2, 3, 4));    // 10
```

Rule of thumb:

- `...` on the **right side** of `=` or in a function call → **spread** (expands).

- `...` on the **left side** of `=` or in a parameter list → **rest** (collects).

9. Multi-Dimensional Arrays

An array can contain other arrays — useful for grids, matrices, etc.

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(matrix[1][2]); // 6 (row 1, column 2)

// Loop through a 2D array
for (let row of matrix) {
  for (let val of row) {
    console.log(val);
  }
}
```

Part C — Objects

An **object** is a collection of **key-value pairs**. Arrays store ordered data; objects store named data.

1. Creating Objects

```
let person = {
  name: "Aman",
  age: 25,
  city: "Bhopal",
  isStudent: true
};
```

- The text before `:` is the **key** (always a string under the hood).
- The text after `:` is the **value** (any type, including other objects, arrays, functions).

2. Accessing Properties

Two ways:

```
// Dot notation (preferred)
console.log(person.name);    // "Aman"

// Bracket notation
console.log(person["name"]);  // "Aman"
```

When to use brackets:

- The key has spaces or special characters: `person["first name"]`
- The key is in a variable: `let k = "name"; person[k]`

```
let key = "age";
console.log(person[key]);    // 25
console.log(person.key);     // undefined ← looks for literal "key"!
```

3. Adding, Updating, Deleting Properties

```
let person = { name: "Aman", age: 25 };

// Add
person.city = "Bhopal";
person["country"] = "India";

// Update
person.age = 26;
```

```
// Delete
delete person.city;

console.log(person);
// { name: "Aman", age: 26, country: "India" }
```

4. Methods — Functions Inside Objects

When a property's value is a function, we call it a **method**.

```
let calculator = {
  add: function(a, b) {
    return a + b;
  },
  subtract: function(a, b) {
    return a - b;
  }
};

console.log(calculator.add(5, 3)); // 8
console.log(calculator.subtract(10, 4)); // 6
```

Shorthand method syntax (ES6):

```
let calculator = {
  add(a, b) { return a + b; },
  subtract(a, b) { return a - b; }
};
```

The **this** Keyword (brief intro)

Inside a method, **this** refers to the object itself.

```
let person = {
  name: "Aman",
```

```
    greet() {  
        console.log("Hello, I am " + this.name);  
    }  
};  
  
person.greet();    // "Hello, I am Aman"
```

We'll go deep into `this` in **Phase 4** — it has many subtleties. For now, just know it works inside methods.

5. Nested Objects

Objects can contain other objects.

```
let user = {  
    name: "Aman",  
    address: {  
        city: "Bhopal",  
        state: "MP",  
        pincode: 462001  
    },  
    hobbies: ["reading", "coding"]  
};  
  
console.log(user.address.city);    // "Bhopal"  
console.log(user.hobbies[0]);      // "reading"
```

6. Object Destructuring

Extract properties into variables, cleanly.

```
let person = { name: "Aman", age: 25, city: "Bhopal" };  
  
let { name, age } = person;  
console.log(name, age);    // "Aman" 25
```

Rename while destructuring:

```
let { name: fullName, age: years } = person;  
console.log(fullName, years);    // "Aman" 25
```

Default values:

```
let { name, country = "India" } = person;  
console.log(country);    // "India" (since person has no country)
```

Nested destructuring:

```
let user = { name: "Aman", address: { city: "Bhopal" } };  
let { address: { city } } = user;  
console.log(city);    // "Bhopal"
```

7. Spread with Objects

```
let person = { name: "Aman", age: 25 };  
  
// Copy  
let copy = { ...person };  
  
// Combine  
let extra = { city: "Bhopal", country: "India" };  
let combined = { ...person, ...extra };  
// { name: "Aman", age: 25, city: "Bhopal", country: "India" }  
  
// Override  
let updated = { ...person, age: 26 };  
// { name: "Aman", age: 26 }
```

When keys overlap, the **last one wins**.

8. Useful Object Methods

```
let person = { name: "Aman", age: 25, city: "Bhopal" };

console.log(Object.keys(person)); // ["name", "age", "city"]
console.log(Object.values(person)); // ["Aman", 25, "Bhopal"]
console.log(Object.entries(person)); // [["name", "Aman"], ["age", 25], ["city", "Bhopal"]]
```

`Object.entries` is especially useful with array iteration methods:

```
Object.entries(person).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});
```

`Object.assign` — Merge objects (older syntax)

```
let merged = Object.assign({}, person, { age: 26, country: "India" });
```

Modern code prefers `{ ...person, ... }` spread syntax, but you'll see `Object.assign` in older codebases.

`Object.freeze` and `Object.seal`

```
let frozen = Object.freeze({ name: "Aman" });
frozen.name = "Raj"; // silently fails (or throws in strict mode)
console.log(frozen.name); // "Aman"

let sealed = Object.seal({ name: "Aman" });
```

```
sealed.name = "Raj";           // ✅ can modify existing
sealed.age = 25;                // ❌ cannot add new properties
```

- `freeze` — fully immutable.
- `seal` — can modify existing properties, but can't add or delete.

9. Looping Through Objects

`for...in` loop

```
let person = { name: "Aman", age: 25, city: "Bhopal" };

for (let key in person) {
  console.log(key, ":", person[key]);
}
```

`Object.keys` with `forEach`

```
Object.keys(person).forEach(key => {
  console.log(key, person[key]);
});
```

`Object.entries` with destructuring (cleanest)

```
for (let [key, value] of Object.entries(person)) {
  console.log(key, ":", value);
}
```

10. Objects vs Arrays — When to Use Which?

Use an array when...	Use an object when...
Order matters	You're describing a single thing
Items are similar (a list)	Each property has a different meaning

Use an array when...	Use an object when...
You'll loop through all items	You'll access specific named values

Examples:

- A list of usernames → array
- A single user's profile → object
- A list of users (each user is an object) → array of objects (very common!)

```
let users = [
  { name: "Aman", age: 25 },
  { name: "Priya", age: 30 },
  { name: "Raj", age: 22 }
];

// Get all names
let names = users.map(u => u.name); // ["Aman", "Priya", "Raj"]

// Filter adults
let adults = users.filter(u => u.age >= 25);

// Average age
let avgAge = users.reduce((sum, u) => sum + u.age, 0) / users.length;
```

This pattern — **array of objects** — is the backbone of almost every real application. Master it.

Part D — Mini Projects

These use everything from Phase 1 and Phase 2 together. Encourage students to attempt all of them.

Project 1: To-Do List (in-memory)

```

let todos = [];

function addTodo(task) {
  todos.push({ id: todos.length + 1, task, done: false });
}

function completeTodo(id) {
  let todo = todos.find(t => t.id === id);
  if (todo) todo.done = true;
}

function removeTodo(id) {
  todos = todos.filter(t => t.id !== id);
}

function showTodos() {
  todos.forEach(t => {
    console.log(`${t.id}. [${t.done ? "x" : " "}] ${t.task}`);
  });
}

addTodo("Learn JS");
addTodo("Build a project");
addTodo("Sleep");
completeTodo(1);
showTodos();
// 1. [x] Learn JS
// 2. [ ] Build a project
// 3. [ ] Sleep

```

Project 2: Student Grade Tracker

```

let students = [
  { name: "Aman", marks: [80, 90, 75] },
  { name: "Priya", marks: [95, 85, 92] },
  { name: "Raj", marks: [60, 55, 70] }
];

function getAverage(marks) {
  return marks.reduce((sum, m) => sum + m, 0) / marks.length;
}

function getGrade(avg) {
  if (avg >= 90) return "A";
  if (avg >= 75) return "B";
  if (avg >= 60) return "C";
  return "F";
}

students.forEach(s => {
  let avg = getAverage(s.marks);
  console.log(`${s.name}: Avg = ${avg.toFixed(2)}, Grade = ${getGrade(avg)}`);
});

```

Project 3: Shopping Cart Logic

```

let cart = [];

function addItem(name, price, qty = 1) {
  let existing = cart.find(item => item.name === name);
  if (existing) {
    existing.qty += qty;
  } else {
    cart.push({ name, price, qty });
  }
}

```

```

    }
}

function removeItem(name) {
    cart = cart.filter(item => item.name !== name);
}

function getTotal() {
    return cart.reduce((sum, item) => sum + item.price * item.qty, 0);
}

function showCart() {
    cart.forEach(item => {
        console.log(`${item.name} x${item.qty} = ₹${item.price * item.qty}`);
    });
    console.log(`Total: ₹${getTotal()}`);
}

addItem("Notebook", 50, 2);
addItem("Pen", 10, 5);
addItem("Notebook", 50); // adds to existing
showCart();

```

Project 4: Word Frequency Counter

```

function wordFrequency(text) {
    let words = text.toLowerCase().split(/\s+/);
    let freq = {};

    words.forEach(word => {
        freq[word] = (freq[word] || 0) + 1;
    });
}

```

```

    return freq;
}

let text = "the quick brown fox jumps over the lazy dog the fox is quick";
console.log(wordFrequency(text));
// { the: 3, quick: 2, brown: 1, fox: 2, jumps: 1, over: 1, lazy: 1, dog: 1, is: 1 }

```

Project 5: Library Management System

Combines arrays, objects, and functions beautifully.

```

let library = {
  books: [],

  addBook(title, author) {
    this.books.push({
      id: this.books.length + 1,
      title,
      author,
      borrowed: false
    });
  },

  borrowBook(id) {
    let book = this.books.find(b => b.id === id);
    if (!book) return "Book not found";
    if (book.borrowed) return "Already borrowed";
    book.borrowed = true;
    return `You borrowed "${book.title}"`;
  },

  returnBook(id) {
    let book = this.books.find(b => b.id === id);
    if (!book) return "Book not found";
  }
}

```

```

        book.borrowed = false;
        return `You returned "${book.title}"`;
    },

    availableBooks() {
        return this.books.filter(b => !b.borrowed);
    }
};

library.addBook("Atomic Habits", "James Clear");
library.addBook("Deep Work", "Cal Newport");
console.log(library.borrowBook(1));
console.log(library.availableBooks());

```

Phase 2 Checklist

By the end of this phase, your students should be able to:

- ☐ Write a function declaration, function expression, and arrow function for the same task.
- ☐ Explain the difference between parameters and arguments.
- ☐ Use default and rest parameters confidently.
- ☐ Use early return to flatten conditional logic.
- ☐ Pass a function as an argument to another function (callback).
- ☐ Explain the difference between mutating and non-mutating array methods.
- ☐ Use `map`, `filter`, `reduce`, `find`, `some`, `every` without referring to documentation.
- ☐ Destructure both arrays and objects, including with renaming and defaults.
- ☐ Use spread and rest in both arrays and objects.
- ☐ Create, access, update, and delete object properties.
- ☐ Loop through an object using `Object.keys/values/entries`.
- ☐ Work with arrays of objects (the bread and butter of real apps).

- ☐ Solve all 5 mini projects without copying.
-

Common Beginner Mistakes to Watch For

1. **Confusing `slice` and `splice`**. Slice is non-mutating; splice is mutating. Different beasts.
2. **Forgetting that `forEach` doesn't return anything**. Students try `let result = arr.forEach(...)` and get `undefined`.
3. **Trying `sort()` on numbers without a compare function**. `[10, 2, 1].sort()` gives `[1, 10, 2]`.
4. **Using dot notation when the key is in a variable.**

```
let k = "name";
person.k    // ❌ looks for literal "k"
person[k]   // ✅
```

5. **Mutating an object/array when they meant to copy.**

```
let copy = original;    // ❌ same reference
let copy = [...original]; // ✅ new copy (shallow)
```

6. **Forgetting the initial value in `reduce`**. Without it, the first element becomes the initial accumulator — usually not what you want.
 7. **Using `for...in` on arrays**. It works but gives weird results. Use `for...of` for arrays, `for...in` for objects.
-